

Data Structure Trees

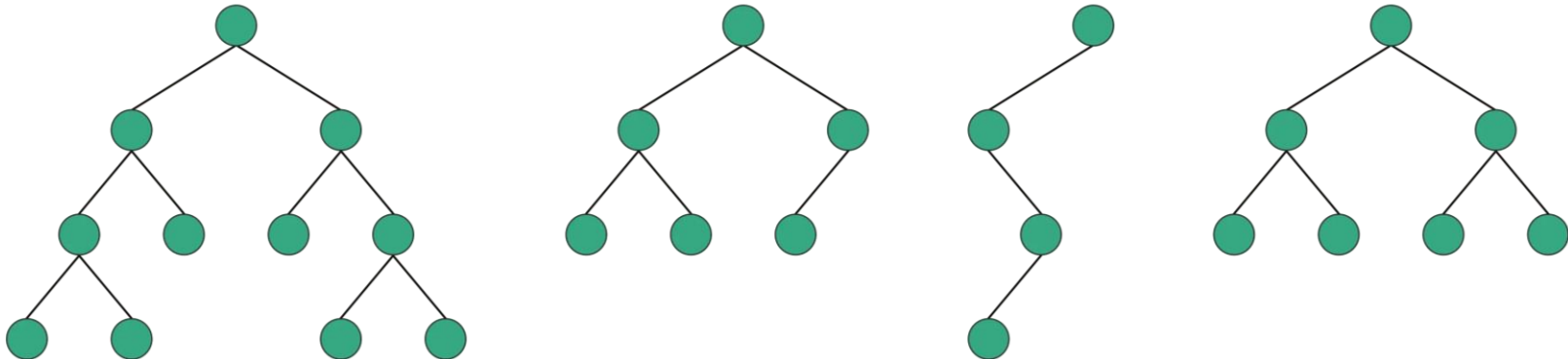
Binary Tree Linked Implementation

Abdul Mateen
Assistant Professor
Department of Computer Science, PU

Binary Tree

A binary tree is a tree, where nodes have only zero, one or two child nodes.

There are four examples of different binary trees. In each of them, nodes have either zero, one or two nodes only.

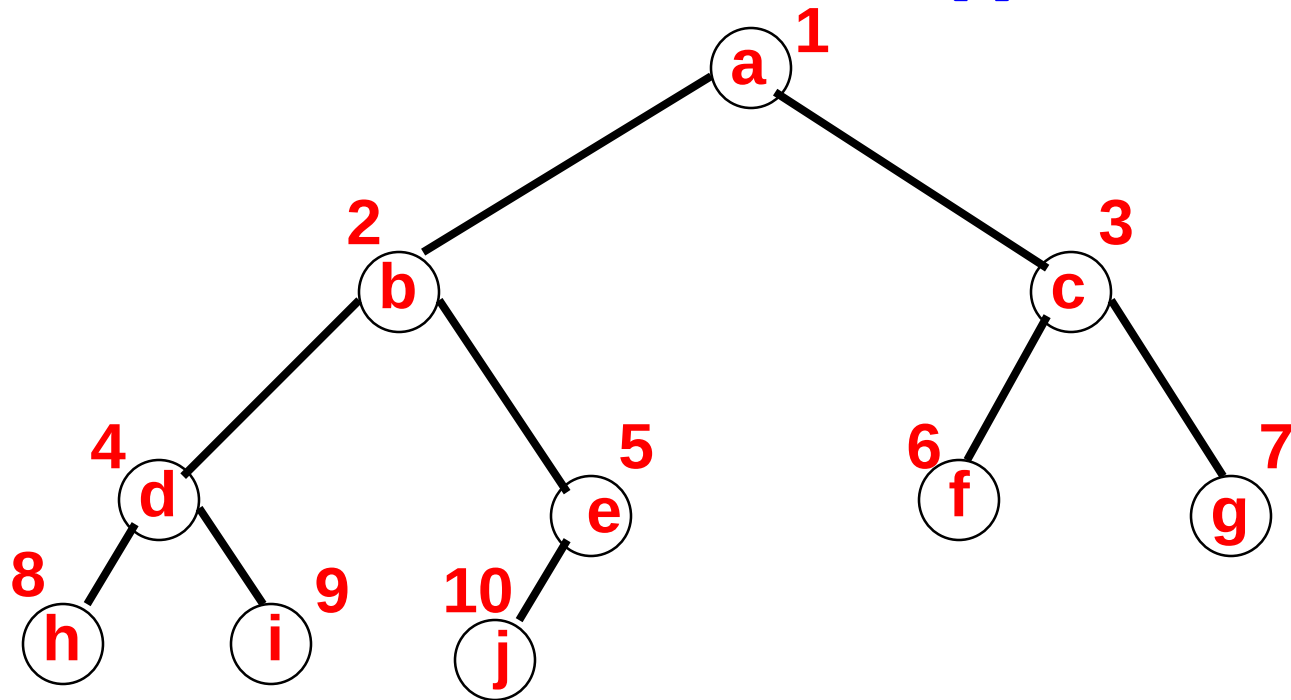


Binary Tree Representation

- Array representation.
- Linked representation.

Array Representation

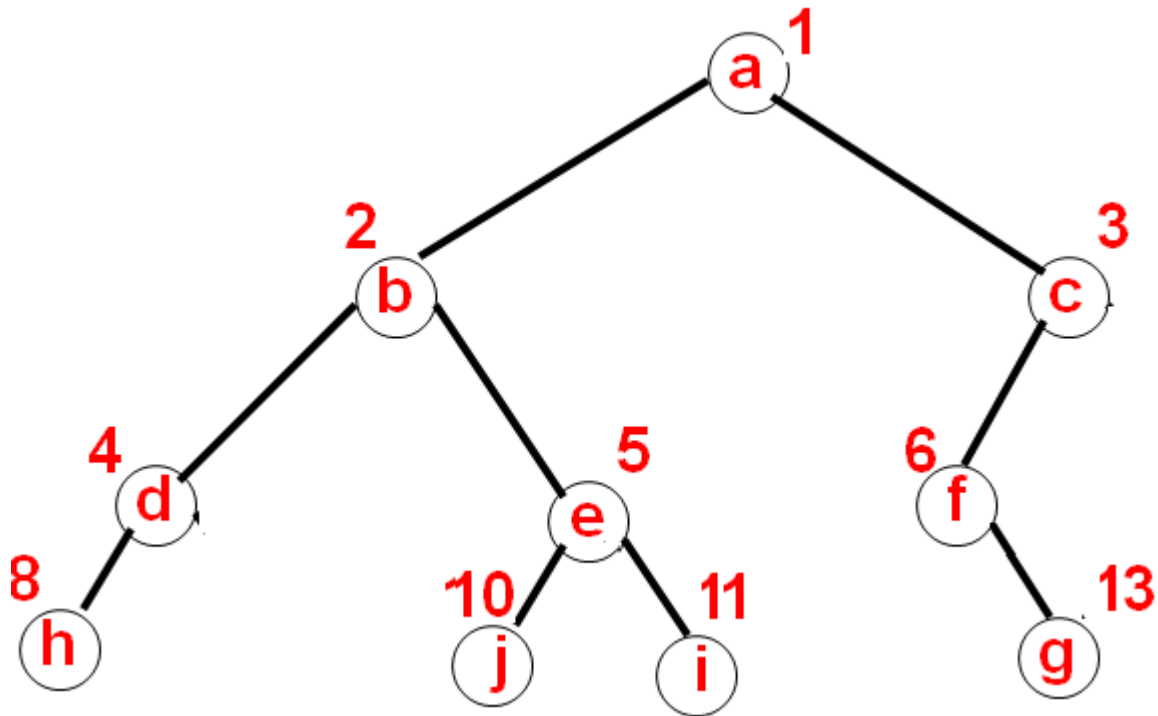
- Number the nodes using the numbering scheme for a full binary tree. The node that is numbered i is stored in `tree[i]`.



tree[]

	a	b	c	d	e	f	g	h	i	j
0					5					10

Array Representation Example



tree[]

	a	b	c	d	e	f		h		j	i		g
0	1	2	3	4	5	6	7	8	9	10	11	12	13

Linked Representation

- Each binary tree node is represented as an object whose data type is `BinaryTreeNode`.
- The space required by an n node binary tree is $n * (\text{space required by one node})$.

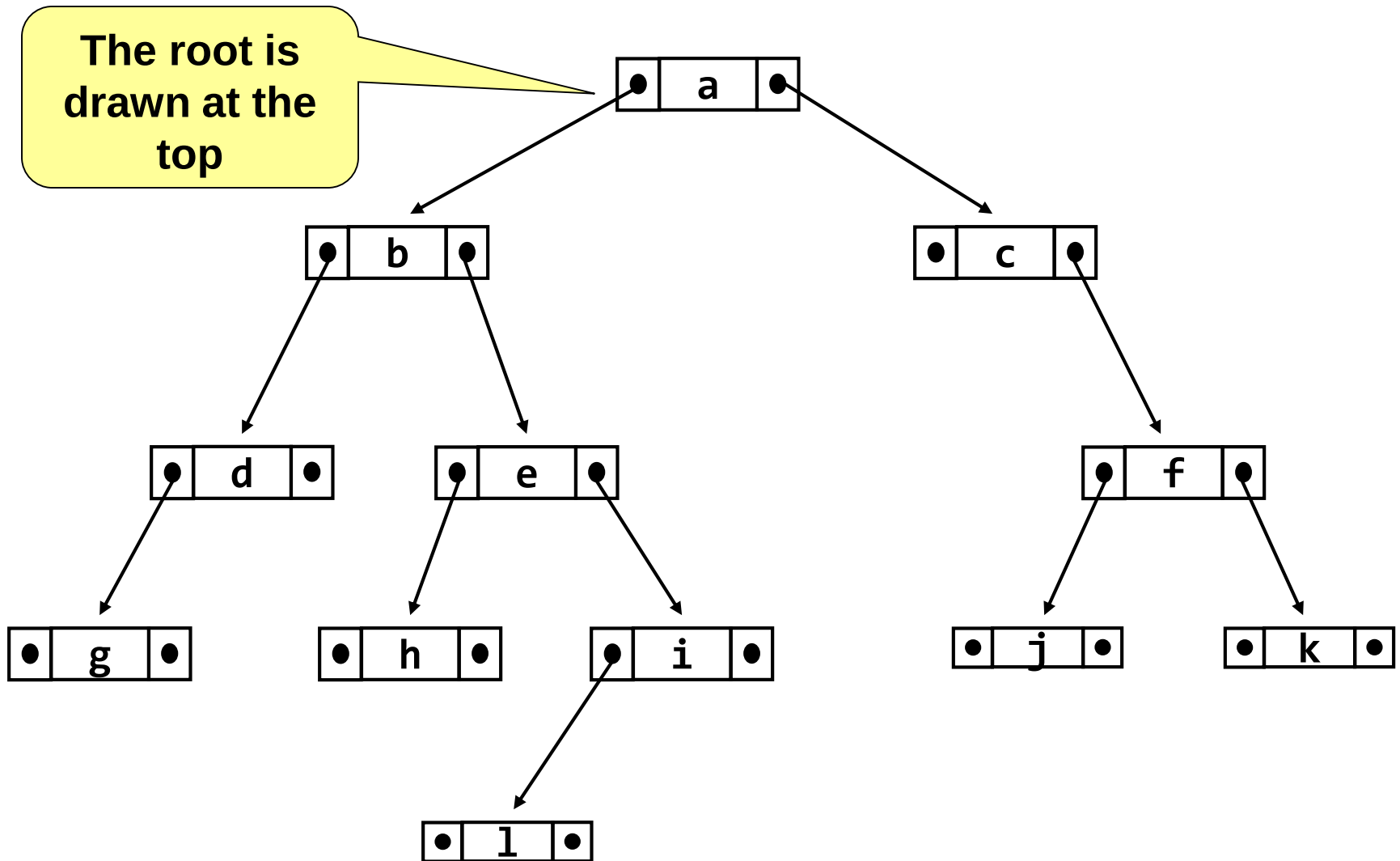
Binary Tree Node Definition

```
struct TNode{  
    int data;  
    TNode *leftChild;  
    TNode *rightChild;  
};
```

Class Binary Tree

```
class BTree{
    TNode *root;
public:
    void insert(int);
    void preOrder();
    void inOrder();
    void postOrder();
    void generateTree();
    ...
};
```


Picture of a binary tree

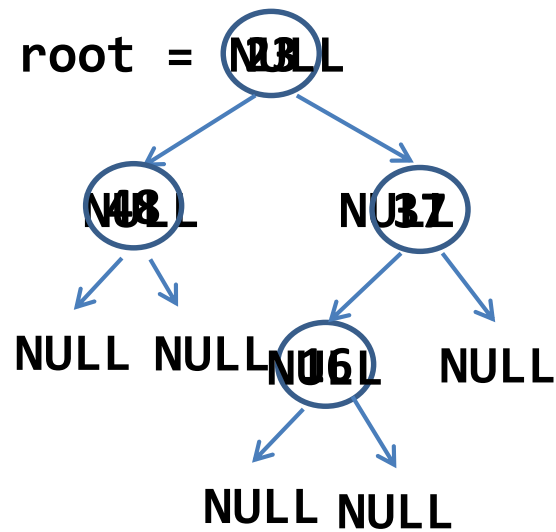


Insert Node

```
void insert(int value){
    root = insert(value, root);
}

BTNode* insert(int value, BTNode *temp){
    if (!temp) return new BTNode(value);
    int randomN = rand() % 2;
    if (randomN == 0)
        temp->left=insert(value, temp->left);
    else
        temp->right=insert(value,temp->right);
    return temp;
}
```

Binary Tree Insertion Example



insert

23

37

16

48

Pre Order Traversal

```
void preorder(){ preorder(root); }
```

```
void preorder(BTNode *temp){  
    if (!temp)    return;  
    cout << temp -> val << ' ';  
    preorder(temp->left);  
    preorder(temp->right);  
}
```

In Order Traversal

```
void inorder(){    inorder(root); }
```

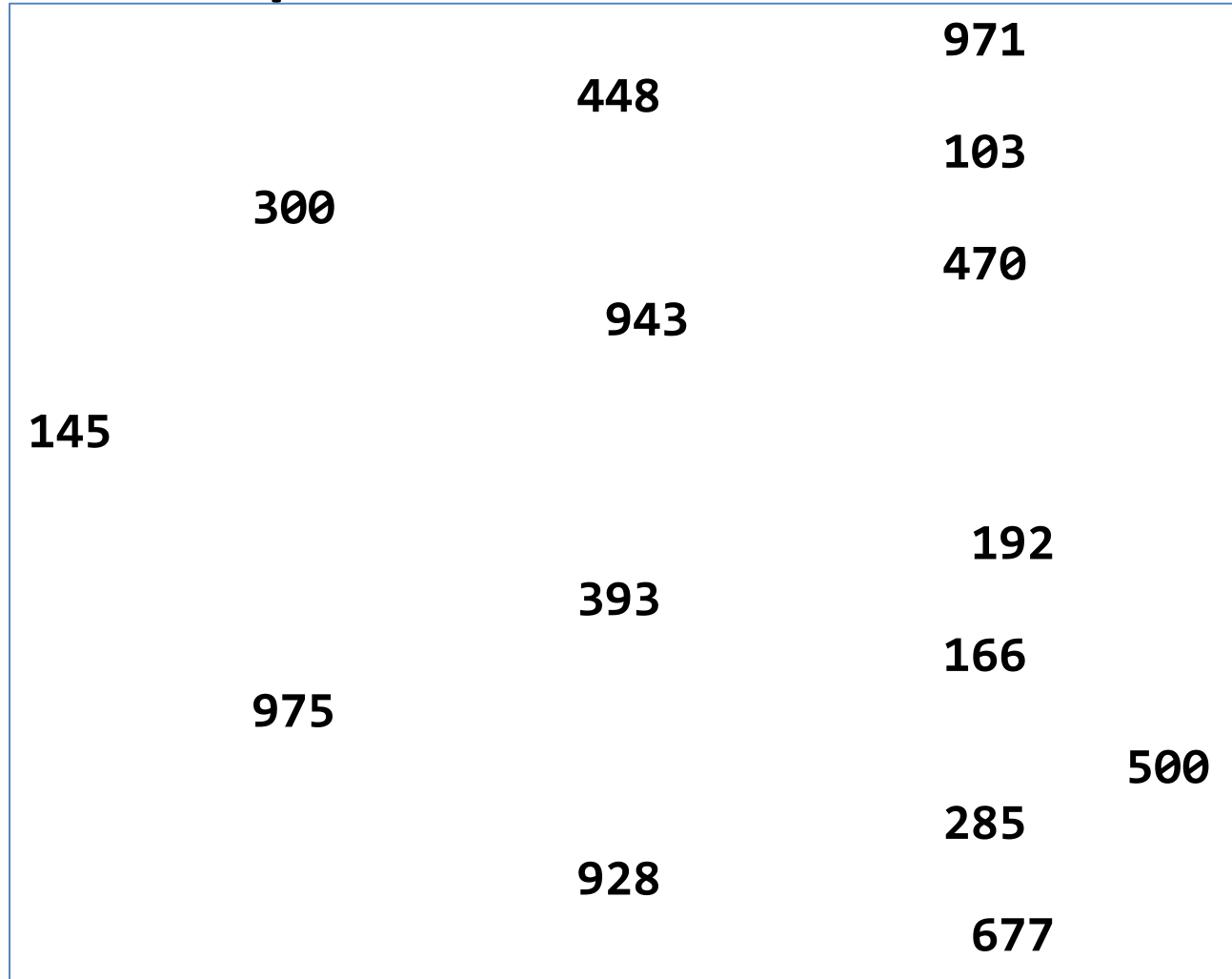
```
void inorder(BTNode *temp){  
    if (!temp)    return;  
    inorder(temp->left);  
    cout << temp -> val << ' ';  
    inorder(temp->right);  
}
```

Generate Tree

```
void generateTree(){  
    generateTree(root);  
}
```

```
void generateTree(BTNode *temp, int tabs = 0){  
    if (!temp)    return;  
    generateTree(temp->right, tabs + 1);  
    for (int i = 0 ; i < tabs ; i++)  
        cout << '\t';  
    cout << temp->val << '\n';  
    generateTree(temp->left, tabs + 1);  
}
```

Output of Generate Tree



Pre Order: 145 975 928 677 285 500 393 166 192 300 943 470 448 103 971

In Order: 677 928 285 500 975 166 393 192 145 943 470 300 103 448 971

Count Nodes

```
int countNodes(){  
    return countNodes(root);  
}
```

```
int countNodes(BTNode *temp){  
    if (!temp)        return 0;  
    int countLeft = countNodes(temp->left);  
    int countRight= countNodes(temp->right);  
    return countLeft + countRight + 1;  
}
```


Find Height

```
int height (){  
    return height(root);  
}
```

```
int height(BTNode *temp){  
    if (!temp) return 0;  
    return max(height(temp->left),  
               height(temp->right)) + 1;  
}
```